

Sparsity-preserving gradient utility tools for PyTorch

Theodore Barfoot¹, Ben Glocker², and Tom Vercauteren¹

¹ King's College London, London, UK ² Imperial College London, London, UK

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

The `torchsparsegradutils` package provides gradient-preserving sparse tensor operations for PyTorch (Paszke et al., 2019), addressing the critical limitation that PyTorch's native sparse operations do not support sparse gradients during backpropagation. This package enables memory-efficient optimization of high-dimensional models by maintaining sparsity patterns throughout the entire forward and backward pass computation.

Key features include: (1) memory-efficient sparse matrix multiplication with sparse gradient preservation, (2) sparse triangular and generic linear system solvers with multiple algorithmic backends (BICGSTAB, CG, LSMR, MINRES), (3) cross-platform sparse solver wrappers for CuPy (Okuta et al., 2017) and JAX (Bradbury et al., 2018), (4) sparse multivariate normal distributions with LL^T and LDL^T parameterizations supporting millions of dimensions, and (5) specialized encoders for spatial neighborhood relationships in volumetric data.

The package addresses PyTorch issue #41128 by implementing custom autograd functions that preserve sparsity in gradients, enabling practical training of models with sparse covariance structures on high-dimensional data where dense alternatives become computationally intractable.

Statement of need

Sparse tensors are essential for computationally tractable machine learning on high-dimensional data, yet PyTorch's sparse tensor operations suffer from a critical limitation: gradients are computed in dense format even when the forward pass maintains sparsity. This results in prohibitive memory usage that scales quadratically with problem dimension rather than linearly with the number of non-zero elements.

For applications requiring sparse covariance modeling—such as medical imaging with millions of voxels, spatial statistics, and large-scale Gaussian processes—dense gradient computation renders optimization infeasible. A sparse covariance matrix with 1 million dimensions and 0.1% sparsity contains 1 billion non-zero elements, but dense gradients would require storing 1 trillion parameters.

This package solves this fundamental limitation by implementing custom autograd functions that preserve sparsity patterns throughout both forward and backward passes. Our sparse multivariate normal distribution enables optimization of million-dimensional Gaussian models with memory usage scaling as $O(nnz)$ rather than $O(n^2)$, where nnz is the number of non-zero elements and n is the dimension.

Beyond memory efficiency, the package addresses the algorithmic challenge of sparse linear system solving by providing multiple iterative solver backends with automatic differentiation support. This enables end-to-end optimization of complex probabilistic models that would be computationally intractable with existing PyTorch sparse operations.

40 Mathematics

41 Sparse Matrix Operations

42 Sparse Matrix Multiplication with Gradient Preservation

43 The package implements sparse matrix multiplication $C = AB$ where $A \in \mathbb{R}^{m \times n}$ is sparse
44 and $B \in \mathbb{R}^{n \times p}$ is dense. The forward pass uses PyTorch's native `torch.sparse.mm`, but the
45 backward pass is reimplemented to preserve sparsity patterns.

46 For sparse matrix A with indices $(A_{\text{rows}}, A_{\text{cols}})$ and values A_{values} , the gradient with respect
47 to A 's values is computed as:

$$\frac{\partial L}{\partial A_{\text{values}}} = \left(\frac{\partial L}{\partial C} [A_{\text{rows}}, :] \odot B [A_{\text{cols}}, :] \right) \mathbf{1}$$

48 where $\odot$ denotes element-wise multiplication, and $\mathbf{1}$ is a summation vector. This selective
49 computation maintains $O(\text{nnz})$ memory complexity for gradients rather than $O(mn)$.

50 Sparse Linear System Solvers

51 The package provides multiple approaches for solving sparse linear systems $Ax = b$:

52 **Triangular Systems:** For triangular matrices $T \in \mathbb{R}^{n \times n}$, we use forward/backward substitution.
53 The gradient computation during backpropagation follows:

$$\frac{\partial L}{\partial b} = T^{-T} \frac{\partial L}{\partial x}, \quad \frac{\partial L}{\partial T_{\text{values}}} = - \left(T^{-T} \frac{\partial L}{\partial x} \right) \otimes x^T$$

54 where $\otimes$ denotes the outer product restricted to non-zero indices of T .

55 **Generic Iterative Solvers:** For general sparse systems, we implement iterative methods including
56 Conjugate Gradient (CG), Biconjugate Gradient Stabilized (BICGSTAB), Least Squares Minimal
57 Residual (LSMR), and Minimal Residual (MINRES). These methods solve $Ax = b$ iteratively
58 while maintaining differentiability through the implicit function theorem.

59 Sparse Multivariate Normal Distributions

60 The package implements sparse multivariate normal distributions $\eta \sim \mathcal{N}(\mu, \Sigma) \equiv \mathcal{N}(\mu, \Omega^{-1})$
61 with two parameterizations for efficient sampling from $\epsilon \sim \mathcal{N}(\mathbf{0}, I)$:

62 **LDL^T Parameterization:** Using sparse unit lower triangular matrices L and diagonal D :

$$\eta_{\Sigma} = \mu + LD^{1/2}\epsilon, \quad \eta_{\Omega} = \mu + L_{\Omega}^{-T}D_{\Omega}^{-1/2}\epsilon$$

63 **LL^T Parameterization:** Using sparse lower triangular matrices L with positive diagonals:

$$\eta_{\Sigma} = \mu + L_{\Sigma}\epsilon, \quad \eta_{\Omega} = \mu + L_{\Omega}^{-T}\epsilon$$

64 The LDL^T parameterization provides superior numerical stability for precision matrices by
65 avoiding strict positive definiteness constraints.

66 Acknowledgements

67 The authors acknowledge the PyTorch development team for providing the foundational
68 sparse tensor infrastructure. We thank the SciPy (Virtanen et al., 2020), CuPy (Okuta et
69 al., 2017), and JAX (Bradbury et al., 2018) communities for high-performance sparse linear
70 algebra implementations. Algorithm implementations adapt and extend methods from pykrylov
71 (BICGSTAB), cornellius-gp/linear_operator (CG, MINRES), and pytorch-minimize (LSMR)
72 (Saad, 2003). This work was supported by the Wellcome/EPSRC Centre for Interventional
73 and Surgical Sciences and the School of Biomedical Engineering & Imaging Sciences, King's
74 College London.

75 References

- 76 Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G.,
77 Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). JAX: Composable
78 transformations of python+NumPy programs. *GitHub Repository*.
- 79 Okuta, R., Unno, Y., Nishino, D., Hido, S., & Loomis, C. (2017). CuPy: A NumPy-compatible
80 library for NVIDIA GPU calculations. *Proceedings of Workshop on Machine Learning
81 Systems (LearningSys) in The Thirty-First Annual Conference on Neural Information
82 Processing Systems (NIPS)*.
- 83 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin,
84 Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-
85 performance deep learning library. *Advances in Neural Information Processing Systems*, 32,
86 8024–8035.
- 87 Saad, Y. (2003). *Iterative methods for sparse linear systems* (2nd ed.). SIAM.
- 88 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
89 Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0:
90 Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3),
91 261–272.